# Práctica 7.

• **El Zen de Python (PEP 20):** ¿Qué es el PEP 20 o 'Zen de Python' y cómo influyen los aforismos dentro de este listado dentro de la programación.

¡Claro! El Zen de Python, o PEP 20, es una colección de 19 aforismos escritos por Tim Peters que encapsulan la filosofía de diseño del lenguaje Python. Puedes acceder a ellos escribiendo `import this` en cualquier intérprete de Python.

Estos aforismos no son reglas estrictas, sino principios que buscan guiar a los desarrolladores hacia la creación de código más legible, simple y mantenible. Influyen en la programación de varias maneras clave:

- **Legibilidad y claridad**: Principios como "Bonito es mejor que feo" y "Explícito es mejor que implícito" fomentan un código fácil de entender. Esto facilita la colaboración en equipo y el mantenimiento a largo plazo.
- **Simplicidad**: "Simple es mejor que complejo" impulsa a los programadores a buscar soluciones directas y claras, evitando complejidades innecesarias.
- **Consistencia**: "Debe haber una —y preferiblemente solo una— manera obvia de hacerlo" busca establecer patrones comunes para resolver problemas, lo que hace que el código de diferentes desarrolladores sea más homogéneo y fácil de seguir.
- **Manejo de errores**: "Los errores nunca deben pasar silenciosamente" subraya la importancia de manejar las excepciones de forma explícita, evitando que los problemas se oculten.

En resumen, el Zen de Python es una brújula filosófica que ayuda a los desarrolladores a escribir código que no solo funciona, sino que también es elegante, comprensible y fácil de mantener [7VTC, 47wW].

En sintesis, son principios que buscan guiar a los desarrolladores a la creación y diseño de código mas legible, simple y mantenible.

• **La Guía de Estilo PEP 8:** ¿Por qué es fundamental seguir el estándar PEP 8 en

proyectos colaborativos y cuáles son sus reglas mas importantes? Proporciona un ejemplo breve.

### ¿Por qué es fundamental seguir el estándar PEP 8 en proyectos colaborativos?

Seguir el PEP 8 es clave para que el código sea:

- **Legible y comprensible:** Imagina que cada desarrollador usa un estilo diferente. ¡Sería un caos! El PEP 8 unifica el estilo, haciendo que el código sea más fácil de leer y entender para todos, sin importar quién lo haya escrito.
- **Consistente:** Asegura que todo el código en un proyecto tenga una apariencia uniforme, lo que reduce la fatiga mental al cambiar de un archivo a otro o al leer el trabajo de un colega.
- **Fácil de mantener:** Cuando el código es consistente y legible, es mucho más sencillo encontrar errores, aplicar actualizaciones o añadir nuevas funcionalidades. Esto ahorra tiempo y esfuerzo a largo plazo.

### Reglas más importantes del PEP 8

Aunque tiene muchas reglas, aquí te dejo algunas de las más fundamentales y que más impacto tienen:

- **Indentación:** Usa 4 espacios por nivel de indentación. ¡Nunca uses tabulaciones! Es una de las reglas de oro.
- **Longitud de línea:** Limita todas las líneas a un máximo de 79 caracteres. Esto facilita la lectura en diferentes editores y herramientas.
- **Nombres de variables y funciones:**
  - Usa `snake_case` (todo en minúsculas y palabras separadas por guiones bajos) para nombres de funciones, variables y métodos.
  - Para nombres de clases, usa `CamelCase` (la primera letra de cada palabra en mayúscula, sin guiones).
  - Para constantes, usa `ALL_CAPS` (todo en mayúsculas, palabras separadas por guiones bajos).
- **Espacios en blanco:**
  - Siempre pon un espacio alrededor de operadores (asignación, comparación, aritméticos).
  - Evita espacios en blanco innecesarios justo dentro de paréntesis, corchetes o llaves.
  - Deja dos líneas en blanco para separar definiciones de funciones y clases de nivel superior, y una línea en blanco para separar métodos dentro de una clase o bloques lógicos de código.
- **Comentarios:** Mantén los comentarios actualizados y asegúrate de que sean claros y concisos. No comentes lo obvio, sino el "por qué" de las cosas.
- **Importaciones:** Coloca las importaciones al principio del archivo, una por línea y en orden alfabético. Separa las importaciones estándar, de terceros y locales con líneas en blanco.

## Ejemplo breve

Aquí tienes un pequeño fragmento de código que sigue algunas de estas reglas del PEP 8:

```
# Importaciones estándar
import os
import sys
# Importaciones de terceros
import requests
# Constante
MAX_RETRIES = 3
class UserManager:
    """Gestiona usuarios en el sistema."""
    def __init__(self, database_connection):
        self.db = database_connection
    def get_user_by_id(self, user_id):
        """
        Obtiene un usuario por su ID de la base de datos.
        """
        if not isinstance(user_id, int) or user_id <= 0:
            raise ValueError("ID de usuario inválido.")
        # Realiza la consulta a la base de datos
        user_data = self.db.query(f"SELECT * FROM users WHERE
id = {user_id}")
        return user_data
def send_welcome_email(user_email, username):
    """
    Envía un correo electrónico de bienvenida a un nuevo usua
rio.
    """
    subject = "¡Bienvenido a nuestra plataforma!"
    body = f"Hola {username},\n\nGracias por registrarte."
    # Lógica para enviar el email...
    print(f"Enviando email a {user_email}: '{subject}'")
if __name__ == "__main__":
```

```
manager = UserManager("my_db_conn")
user = manager.get_user_by_id(123)
if user:
    send_welcome_email("test@example.com", "JuanPerez")
else:
    print("Usuario no encontrado.")
```

- **Código Lineal vs. Legibilidad:** ¿Cómo influye el uso de operadores ternarios, funciones lambda y comprensiones de listas/diccionarios dentro de la filosofía de Python? ¿En qué punto estas herramientas de código lineal dejan de ser 'legibles' y rompen con el principio de esta filosofía y guía de estilo de código?

En general, los operadores ternarios, las funciones lambda y las comprensiones de listas/diccionarios se consideran herramientas muy potentes y "pythónicas" para escribir código conciso y expresivo. Se alinean con la idea de "menos es más" y de tener código que sea casi como pseudocódigo.

Aquí te cuento cómo influyen y dónde podrían empezar a tambalearse:

- **Operadores Ternarios (`a if condition else b`)**:
  - **Influencia:** Son geniales para asignar un valor basado en una condición simple en una sola línea. Hacen el código más compacto y a menudo más legible que un `if/else` de varias líneas cuando la lógica es sencilla. Evitan la necesidad de declarar una variable y luego reasignarla.
  - **Punto de ruptura:** Cuando la condición o las expresiones para `a` y `b` se vuelven demasiado complejas. Si tienes múltiples condiciones anidadas (`a if cond1 else (b if cond2 else c)`), o si las expresiones son llamadas a funciones largas, el ojo tiene que saltar mucho y la legibilidad se pierde rápidamente. El principio de "la legibilidad cuenta" se rompe cuando una línea de código se convierte en un desafío mental para entender.
- **Funciones Lambda (`lambda args: expression`)**:
  - **Influencia:** Son perfectas para funciones pequeñas, anónimas y de un solo uso, especialmente cuando se pasan como argumentos a funciones de orden superior como `map()`, `filter()`, `sorted()`, o para callbacks simples. Promueven el estilo funcional y mantienen el código compacto al no requerir una definición `def` formal.
  - **Punto de ruptura:** Una lambda debe ser una expresión. Si necesitas usar sentencias (como `if/else` complejos, bucles, asignaciones de variables) o si la lógica requiere múltiples líneas, una lambda se convierte en un hack ilegible o simplemente imposible de implementar de manera limpia. Si tu lambda ocupa más espacio del que ocuparían unas pocas líneas de una función `def` clara con un nombre descriptivo, ya no es pythónica. Si requiere comentarios para entenderse, definitivamente es demasiado compleja.

- **Comprensiones de Listas/Diccionarios (`[expr for item in iterable if condition]`):**
  - **Influencia:** Son una de las características más amadas de Python. Permiten crear nuevas listas, diccionarios o conjuntos de forma muy concisa y eficiente. Son mucho más legibles que los bucles `for` equivalentes de varias líneas para transformaciones o filtrados simples. Transmiten la intención de "crear una nueva colección" de manera muy clara.
  - **Punto de ruptura:** Al igual que los ternarios, la complejidad es el enemigo.
    - **Comprensiones anidadas:** Una compresión de doble bucle ( `[expr for x in y for z in w]`) suele ser legible. Una de triple bucle o más es casi siempre una pesadilla.
    - **Lógica compleja:** Si el `expr` o el `condition` contienen operaciones complicadas, llamadas a funciones con efectos secundarios, o si la línea se extiende demasiado, la legibilidad disminuye.
    - **Efectos secundarios:** Las comprensiones deben ser puras (sin efectos secundarios). Si intentas hacer más que simplemente construir la colección (como imprimir algo o modificar otra variable), estás usando la herramienta incorrecta y rompiendo con la filosofía.

En resumen, la clave está en el equilibrio. Python valora la expresividad y la concisión, pero nunca a expensas de la legibilidad. Si el código lineal te obliga a forzar la lógica, a anidar demasiadas cosas o a escribir líneas excesivamente largas que el ojo no puede parsear fácilmente, es una señal clara de que es mejor desglosarlo en pasos más pequeños y explícitos, quizás con un `if/else` tradicional, una función `def` bien nombrada o un bucle `for` convencional.

Considero que es sumamente importante tener presentes lo estándares de estilo y filosofía de algunos lenguajes de programación pues fungen como un tipo de “reglas/guía” a seguir para una correcta lectura, modificación y exploración del código. Por otro lado, pueden parecer un poco engorrosas pues en ciertos casos limita la creatividad y flujo de código de algún programador, sin embargo es un mal necesario para compartir código.